Super Quantization

Alan Tuning
0x123B96D

Abstract

We introduce *Super Quantization*, an aggressive weight quantization strategy applicable to all linear layers in a neural network. In this approach, model weights are constrained to values of the form $\pm 2^k$ (with $k \in \mathbb{Z} \cup \{-\infty\}$), while activations remain in full precision. Unlike conventional low-bit quantization methods that still require standard multiplications, *Super Quantization* replaces those operations with simple exponent additions, thereby challenging the necessity of using multiplications in current GPU architectures. We detail two variants: (1) an int2 quantization that restricts weights to $\{-1, 0, 1, 2\}$ and (2) a boolean quantization with weights in $\{0, 1\}$. Experimental and theoretical results on Natural Language Processing (NLP) tasks indicate that this technique can reduce the total model weight by up to 80% and achieve a near fourfold speedup in matrix multiplications, while maintaining almost the same score as the full precision network. We also analyze how *Super Quantization* scales with increasing model size and provide an in-depth exploration of which network components should be quantized.

1 Introduction

The recent surge in transformer model size, particularly in NLP, has intensified the demand for efficient inference [**1**], especially in resource-constrained devices [**4**]. Although standard quantization methods (e.g., fp8, int8, int4, ...) have been widely adopted to reduce memory footprint and accelerate computations, they still rely on conventional multiplications [**3**] [**2**], which remain computationally costly when performed at scale. Industry trends including Intel's recent initiative toward native int2 computations [**5**] highlight the potential for even lower-bit representations to deliver further efficiency gains.

In response to these challenges, binary neural networks (BNNs) were introduced, drastically reducing the precision of weights and activations to binary values. However, BNNs often suffer from substantial accuracy losses in transformers due to their overly restrictive nature [**6**] [**10**]. To address this, we propose *Super Quantization*, an alternative that generalizes the concept of BNNs. By constraining weights to powers of two, i.e. $\pm 2^k$ with $k \in \mathbb{Z} \cup \{-\infty\}$, we

retain a level of expressivity that is typically lost in full binary models while simultaneously enabling the replacement of multiplications with presumably more efficient operations.

Notably, another recent work [**7**] also highlights an approach that maintains activations in full precision and even explores the potential of sub-bit quantization. They suggest that very little performance loss occurs when using sub-bit representations across a wide range of neural network architectures, including transformer encoders for NLP. However, the absence of specific empirical results for this case suggests that the performance impact in this context may be more significant. The authors further recommend fine-tuning existing models before applying any quantization. Therefore, for the purpose of this study, we focus our analysis on decoder-only transformer used for text generation. To thoroughly assess the advantages of this technique, we train a transformer model from scratch and conduct an in-depth scaling analysis of the method.

2 Super Quantization

2.1 $\mathbf{SQ_2}$ Super Quantization

An intuitive way of simplifying correlations would be to consider the following: two variables can be either positively correlated, negatively correlated, or uncorrelated. Although a natural approach would be to use $\{-1, 0, 1\}$, by adding the value 2 we allow the model to distinguish between weak and strong positive correlations, as multiplying by 2 is equivalent to increment the exponent by one which can be achieved fairly easily with all chips. The primary reason for this add-on remains that it allows one to encode each parameter without any loss of information on 2 bits. Hence, we define the first variant of *Super Quantization* (denoted SQ_2) restricting the weights to the set $\{-1, 0, 1, 2\}$.

Beyond its conceptual appeal, this trivial choice suggests potential hardware advantages as it minimizes the number of logic gates required for multiplications between a floating point and an int2. It can be shown[1] as an undergraduate exercice, that such a circuit can be made out of $5 + m + 2e$ logic gates, where m (respectively e) corresponds to the number of bits affected to mantissa (resp. exponent).

Super Quantization is applied only during the forward pass[2]. Meanwhile, activations, backward activations, and backward weights remain in full precision to ensure stable convergence. We also explore alternative configurations such

[1] the naive circuit found was not latency optimized and had a linear time complexity, but there are evidence that it could be further optimized for this stake

[2] full-precision weights are stored during training and are clamped when performing the forward pass, turning them 'int2' their quantized counterparts

as $\{-2, -1, 0, 1\}$, $\{-2, -1, 1, 2\}$, and $\{-1, 0, 1/2, 1\}$, evaluating their impact on performance and efficiency.

2.2 SQ_1 Super Quantization

The second approach, SQ_1 *Super Quantization* (denoted SQ_1), takes weight reduction even further by limiting weights to a binary set. As in SQ_2, this is also a quantized-aware training method, leaving the activation and the backward process untouched. Here, the model can only choose between 1 (correlation) and 0 (no correlation). Unlike standard binary neural networks (BNNs), which typically use $\{-1, 1\}$ [**8**], we also examine other configurations such as $\{-1, 0\}$ and $\{-1/2, 1/2\}$.

BBNs show promising results in quantum computing, as it has been shown that such computers could at least theoretically perform efficient binary optimization and reach global minima [**9**].

2.3 Learnable Bits

Large Language Models (LLMs) are frequently described as information compressors [**11**], where model complexity is often assessed by simply counting the number of parameters. However, this approach overlooks the actual information capacity of a network under quantization constraints. To address this limitation, we introduce *Learnable Bits*[3] (B), a metric that quantifies the effective bit count required to represent a trained model's weights considering their constrained representation. While many works have implicitly relied on this measure, they often do so without explicitly naming it. Formally, given a weight matrix W where each element is drawn from a discrete set S, the total number of learnable bits corresponds to the entropy of the weight distribution and is defined as:

$$B = \sum_i \log_2 |S_i|$$

where $|S_i|$ represents the number of unique values that the weights of layer i can take. Unlike conventional floating-point storage, which allocates a fixed 32-bit or 16-bit representation per parameter, *Learnable Bits* reflect the true model entropy by accounting for the sparsity and expressivity of the quantized weight space. This aligns with the view of neural networks as information compressors, where quantization effectively serves as a form of lossy compression.

While B provides an upper bound on the effective number of bits effectively used by a model, not all learnable bits contribute equally during inference. We

[3] which has been indirectly referred in other works

define *Active Learnable Bits* (B_{act}) as the subset of learnable bits that participate in meaningful computations, dynamically adapting to input variations. Mathematically, we can estimate B_{act} by weighting B with an activity factor α_i for each layer:

$$B_{\text{act}} = \sum_i \alpha_i \log_2 |S_i|$$

where α_i measures the proportion of the forward passes that uses the weights of the ith layer. In Mixture of Models (MoE), $B_{\text{act}} \ll B_{\text{learn}}$, indicating that a substantial portion of learned weights remains inactive during typical inference. By leveraging both metrics, we gain deeper insights into model efficiency, allowing for the design of architectures that minimize storage and computational cost while preserving expressivity.

2.4 Normalization constant

In order to stabilize training, it is possible to introduce a normalization constant. As shown in the appendix 4, when applying SQ_1 to an entire Multi Layer Perceptron (MLP) block, normalization should be performed at the end of the block using a scaling factor given by:

$$\gamma = \frac{2}{n}\sqrt{\frac{2}{1-\frac{1}{\pi}}}.$$

At first glance, since *Super Quantization* aims to reduce the number of multiplications, introducing such a normalizing factor might seem counterproductive. However, in transformer architectures, MLP layers are typically surrounded by Layer Normalization, which already involves multiplications. Consequently, this normalization constant can be incorporated into the architecture during training and absorbed by the adjacent normalization layers at inference time, ensuring no additional computational overhead.

3 Experiments

3.1 Experimental Setup

To thoroughly evaluate the effectiveness of *Super Quantization*, we conduct a series of experiments on scaling llm (little language models). In our approach, *Super Quantization* is applied selectively while other components remain in full precision, distinguishing our method from prior work, which adopts a more uniform quantization scheme across transformer layers [**10**] [**7**].

Our experiments explore different model sizes, scaling the depth (D) from 1 to 4. The model hyperparameters are the following: the model dimension is set as $32D$, with two attention heads and a feed-forward dimension scaled to

$128D$. Additionally, we integrate Low Rank Adaptation (LoRA) with a rank of $\frac{dim}{4}$. For reference, Llama 3.1 [**12**] follows a similar scaling strategy, where the model dimension is $128D$ and the feed-forward dimension is $512D$. Our setup closely mirrors this scaling, allowing us to assess whether *Super Quantization* scales across all dimensions.

We separately trained models over two sources: a dataset of python code (CodeParrot) [**13**] and a cleaned subset of english Wikipedia [**14**]. Wikipedia's inclusion ensures that our approach remains effective in tasks that require substantial information retention, as it has been shown that general knowledge was stored in the MLP layer of the transformer [**15**].

Regarding training methodology, we employ a batch size of 32 for a total of 5 million training steps and adjust the learning rate of AdamW based on model depth, starting at 0.001 for models having a small number of parameters and decreasing to 0.0002 for deeper ones. The vocabulary size was chosen to 10,000 tokens and a context length of 256, for computational reasons. Standard positional embeddings are used instead of Rotary Position Embedding (RoPE), as it delivered better results overall.

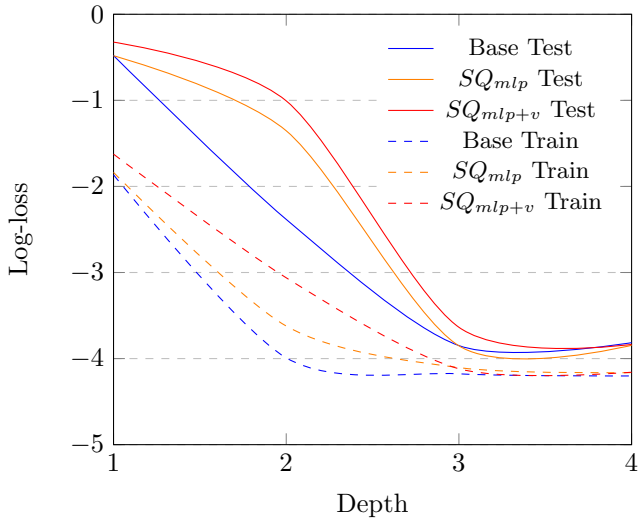
Finally, an additional normalization constant described in section 2.4 is incorporated into the architecture to stabilize training. Our evaluation focuses on several key aspects: scalability, influence of dataset types, effectiveness of different quantization strategies, as well as the role of normalization in stabilizing quantized models.

3.2 Results and Discussion

For each scale, we tested three models on the CodeParrot dataset: a reference non-quantized model (denoted Base), a model quantized using SQ_1 on its MLP layers (denoted SQ_{mlp}), and a model quantized using SQ_1 on its MLP layers together with SQ_2 on its value layers (denoted SQ_{mlp+v}). In particular, we plot the evolution of the log loss for both train and test sets (figure 3.2). As we can deduce from the plot, the log ratio[4] between the train losses of the non-quantized and super-quantized models remains almost constant during the scaling process.

[4] $log(loss_{Base}/loss_{SQ})$

Evolution of the log-loss for different quantization configurations



To provide a clear comparison of the final performance, Table 1 summarizes the final training and test loss values for a network of $D = 4$ across the three quantization configurations. For instance, it suggests that we cannot achieve better performances with a full precision model than with its super-quantized counterpart.

Model Configuration	**Training Loss**	**Test Loss**
Base	**0.0150**	0.0220
SQ_{mlp}	0.0156	**0.0214**
SQ_{mlp+v}	0.0156	0.0216

Table 1: Final loss values for different quantization configurations of a decoder-only transformer with depth 4

It seems that quantizing the entire network is not beneficial. In particular, when using quantized values in -1, 1 on each layer and training from scratch, a network of depth 4 struggled to bring the training loss below 3—reaching 3.22 after 5 million training steps—with the test loss settling at 5.85. This confirms our initial concerns about full quantization, although suboptimal performance might be partly due to our relatively short training duration. During our numerous experiments[5], we found that SQ_{mlp+v} yielded the highest degree of quantization with little to know performance degradation.

[5]We eventually observed that quantizing either the query or key matrix in SQ_1 in addition to the value and feedforward SQ could lead to good performance on bigger models.

On the Wikipedia dataset, we do not observe any significant differences between the Base model and SQ_{mlp} on the deepest architecture. However, the shallowest SQ_{mlp} struggled to reach the performance of the Base model at this depth. This suggests that when the model underfits and the task requires extensive knowledge, quantizing the MLP layers exacerbates the problem, confirming the hypothesis stated in [**15**]. However, when the model is deep enough to properly approximate the task, the performance of SQ_{mlp} remains quite reasonable, even for knowledge-intensive tasks.

Our code achieves an 80% reduction in active learnable bits for SQ_{mlp} and $D = 4$. For instance, in architectures such as Llama 3.1 405B that include three fully connected layers (fc), quantizing only two fc layers allows us to quantize almost two-thirds of the network. In the case of fully quantized fc layers this fraction increases up to 93.60%.

Although incorporating an additional layer normalization leads to a faster convergence and a more significant reduction in training loss, we did not saw a reduction in the test loss. Given the minimal computational cost of normalization, we recommend its inclusion, as it could bring more stability in deeper networks.

Finally, with respect to the optimal quantization values, our experiments did not reveal any superior combination among the alternatives discussed in Sections 2.1 and 2.2. SQ_2 and SQ_1 both offer the simplest of calculations; therefore, we recommend their use to maximize performance.

Overall, super-quantized models show great promise on edge devices. Being both lightweight and optimized, they not only reduce storage requirements by approximately 85% (on a Llama 3.1 405B) but also provide a fourfold improvement in computation speed. This speedup was observed in our C++ implementation[6] when comparing a product of two floating-point matrices with that of a floating-point and a boolean matrix[7].

4 Conclusion and Future Work

In this work, we introduced *Super Quantization*, a novel weight quantization strategy that replaces multiplications with efficient exponent additions. By constraining weights to powers of two, we demonstrated that transformers can achieve significant memory savings and computational speedups while still maintaining near full-precision accuracy. Our key findings highlight that *Super Quantization* scales well and can achieve substantial savings in memory and computation, particularly for on-edge devices.

[6] https://github.com/Degnel/SuperQuantization/blob/master/src/mm.cpp
[7] for a naive $O(n^3)$ algorithm

Our analysis reveals that the most effective approach is a hybrid quantization strategy. The feedforward layers should be prioritized for aggressive quantization (e.g. SQ_1), while the value projection can tolerate a more relaxed scheme (e.g. SQ_2) for additional efficiency gains. However, most of the model's weight footprint already resides in the MLP components, strengthening MLP quantization as the primary lever for efficiency. In contrast, applying *Super Quantization* to query and key projections—especially using SQ_1 or SQ_2—proves detrimental. Additionally, we observe that normalization plays a positive role in mitigating potential degradation arising from quantization. Despite these aggressive quantization strategies, performance degradation remains minimal, even when training on large corpora such as Wikipedia.

For future work, scaling *Super Quantization* to state-of-the-art model sizes and training over extended number of steps could further refine its benefits. Additionally, exploring *sub-bit* quantization exclusively within MLP layers may unlock new efficiency trade-offs. We believe these directions will help push the boundaries of transformer optimization, making them even more enticing for deployment in resource-limited environments.

Appendices

In this section, we rigorously describe the normalization required to stabilize the passage through a SQ_1 layer. To this end, we consider two successive steps: (1) the quantization of activations using a binary matrix Q whose elements are independently drawn from $\{0, 1\}$ (with $\mathbb{P}(Q_{ij} = 0) = \mathbb{P}(Q_{ij} = 1) = \frac{1}{2}$) and (2) the application of a ReLU activation function.

1. Passage through the quantization layer

Let $N \in \mathbb{R}^{m \times n}$ be an activation matrix, where each row represents an input vector with n features, and assume that its elements are independent and normally distributed with zero mean and variance σ^2:

$$N_{il} \sim \mathcal{N}(0, \sigma^2), \quad i = 1, \ldots, m,\ l = 1, \ldots, n.$$

Let $Q \in \{0, 1\}^{n \times k}$ be a binary matrix whose entries are drawn independently from $\{0, 1\}$ with equal probability. We have for every element S_{ij} of $S = NQ \in \mathbb{R}^{m \times k}$:

$$\begin{aligned} Var(S_{ij}) &= Var(\sum_{l=1}^{n} N_{il}\, Q_{lj}) \\ &= \sum_{l=1}^{n} \mathbb{E}[(N_{il}\, Q_{lj})^2] - \big(\mathbb{E}[N_{il}\, Q_{lj}]\big)^2 \\ &= \sum_{l=1}^{n} \mathbb{E}[N_{il}^2 Q_{lj}] - 0 \\ &= \sum_{l=1}^{n} \mathbb{E}[N_{il}^2]\, \mathbb{E}[Q_{lj}] \\ &= \frac{n\sigma^2}{2} \end{aligned}$$

To maintain an output variance equal to σ^2 after quantization, a normalization factor

$$\alpha = \sqrt{\frac{2}{n}}$$

is applied, so that

$$\mathrm{Var}\Big(\alpha\, S_{ij}\Big) = \alpha^2 \cdot \frac{n\,\sigma^2}{2} = \sigma^2.$$

2. Passage through the ReLU activation

We are now interested in computing the variance of the elements of $R = \mathrm{ReLU}(S)$. Let K_j be the number of nonzero terms on column j:

$$K_j = \sum_{l=1}^{n} Q_{lj} \sim \mathrm{Bin}(n, \frac{1}{2}).$$

Given $K_j = k$, we have

$$S_{ij} \mid (K_j = k) \sim \mathcal{N}(0, k\sigma^2).$$

For a normal variable $X \sim \mathcal{N}(0, v)$, it is known that:

$$\mathbb{E}[\mathrm{ReLU}(X)] = \sqrt{\frac{v}{2\pi}},$$
$$\mathbb{E}[\mathrm{ReLU}(X)^2] = \frac{v}{2}.$$

By marginalizing over K_j, we get:

$$\begin{aligned}
\mathbb{E}[\mathrm{ReLU}(S_{ij})] &= \sum_{k=0}^{n} \Pr(K_j = k)\, \mathbb{E}[\mathrm{ReLU}(S_{ij}) \mid K_j = k] \\
&= \frac{\sigma}{\sqrt{2\pi}} \frac{1}{2^n} \sum_{k=1}^{n} \sqrt{k} \binom{n}{k} \\
&= \frac{\sigma}{\sqrt{2\pi}}\, \mathbb{E}\left[\sqrt{K_j}\right]
\end{aligned}$$

$$\begin{aligned}
\mathbb{E}[\mathrm{ReLU}(S_{ij})^2] &= \sum_{k=0}^{n} \Pr(K_j = k)\, \mathbb{E}[\mathrm{ReLU}(S_{ij})^2 \mid K_j = k] \\
&= \frac{\sigma^2}{2}\, \mathbb{E}[K_j] \\
&= \frac{n\,\sigma^2}{4}
\end{aligned}$$

The variance is finally obtained by

$$\begin{aligned}
\mathrm{Var}[R_{ij}] &= \mathbb{E}[R_{ij}^2] - \mathbb{E}[R_{ij}]^2 \\
&= \frac{n\,\sigma^2}{4} - \frac{\sigma^2}{2\pi} \left(\frac{1}{2^n} \sum_{k=1}^{n} \sqrt{k} \binom{n}{k} \right)^2 \\
&= \frac{n\,\sigma^2}{4} - \frac{\sigma^2}{2\pi} \mathbb{E}\left[\sqrt{K_j}\right]^2
\end{aligned}$$

For large n, by the central limit theorem, K_j concentrates around $\frac{n}{2}$ and we can approximate

$$\mathbb{E}\left[\sqrt{K_j}\right] \approx \sqrt{\frac{n}{2}}\,.$$

In this case,

$$\mathrm{Var}[R_{ij}] \approx \frac{n\,\sigma^2}{4} - \frac{\sigma^2}{2\pi}\left(\frac{n}{2}\right) = \frac{n\,\sigma^2}{4}\left(1 - \frac{1}{\pi}\right).$$

Hence, the asymptotic normalization factor can be expressed as:

$$\beta = \frac{2}{\sqrt{n\left(1 - \frac{1}{\pi}\right)}}.$$

3. Overall Normalization in the Feedforward Network

Thus, for a complete forward pass through a hidden layer with a ReLU activation (assuming zero-initialized biases), the output should be normalized by the combined factor:

$$\gamma = \alpha\,\beta = \sqrt{\frac{2}{n}} \times \frac{2}{\sqrt{n\left(1 - \frac{1}{\pi}\right)}} = \frac{2\sqrt{2}}{n\sqrt{1 - \frac{1}{\pi}}}.$$

This normalization ensures that, despite the binary *Super Quantization* and the non-linearity of the ReLU, the activation variance remains stabilized at σ^2.